

UNIVERSITÀ
DEGLI STUDI
DELL'AQUILA

# Lesson 4: Classification and Evaluation Metrics

## Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi dell'Aquila

30 October 2026

# Before we start

- Exercise 3, which we discuss now
- Ridge, Lasso, and the choice of the penalty strength



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Today: from a number to a decision

- Logistic regression: the model and why the cost had to change
- The confusion matrix, and why accuracy lies
- Precision, recall, F1, and the threshold nobody chose
- Receiver operating characteristic (ROC) curves, and where they mislead
- Choosing a threshold from money

# The question has changed

- Lesson 3: given a house, **how many euros?**
- Lesson 4: given a disk drive, **will it fail?**
- Answer with a probability: $0 \leq P(y = 1 \mid x) \leq 1$
- $y = 1$ if the drive failed, $y = 0$ if not; $x$ holds its six counters



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# The fleet

- **8,000 disk drives** in a data centre
- Six SMART counters each: Self-Monitoring, Analysis and Reporting Technology
- Reallocated sectors, spin retries, read and seek error rates, temperature, hours in service
- Label: did it fail within thirty days?
- `train_test_split(..., stratify=y)`: 6,000 to train, 2,000 to test



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

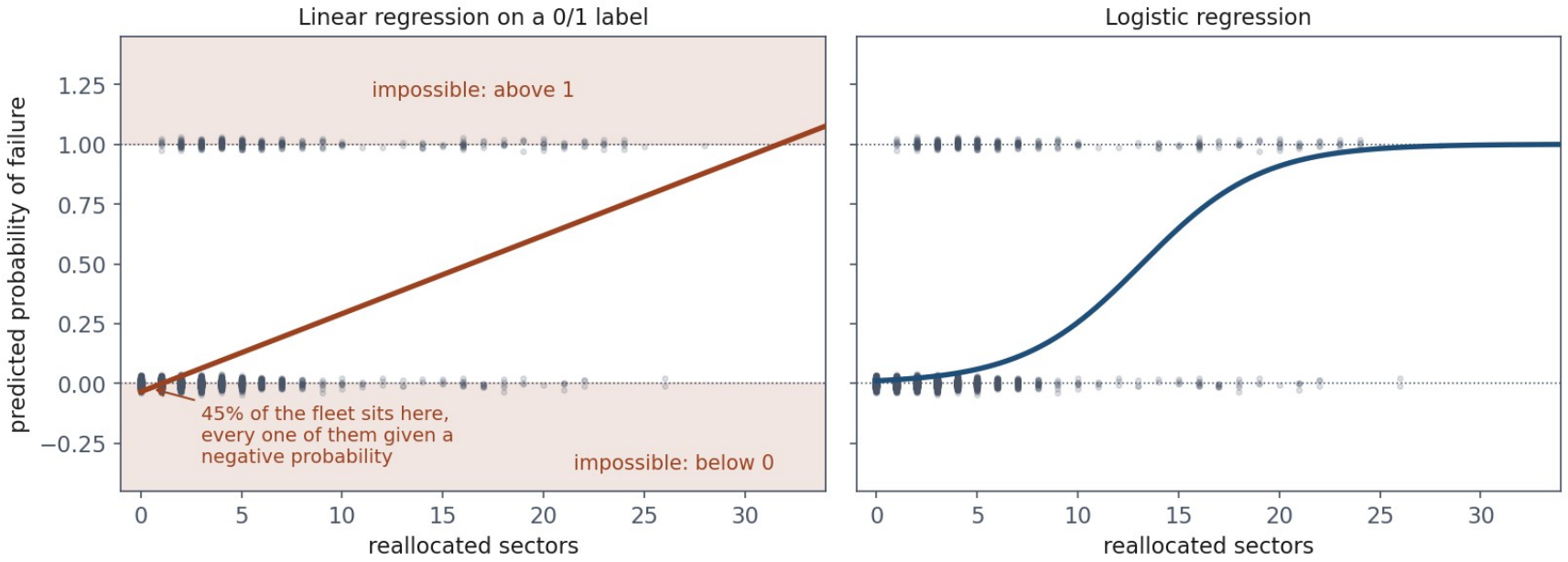
# 306

- **306 of the 8,000 drives failed**: 306 ÷ 8,000 = 3.8% of the fleet
- Which means 100% − 3.8% = 96.2% did not



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

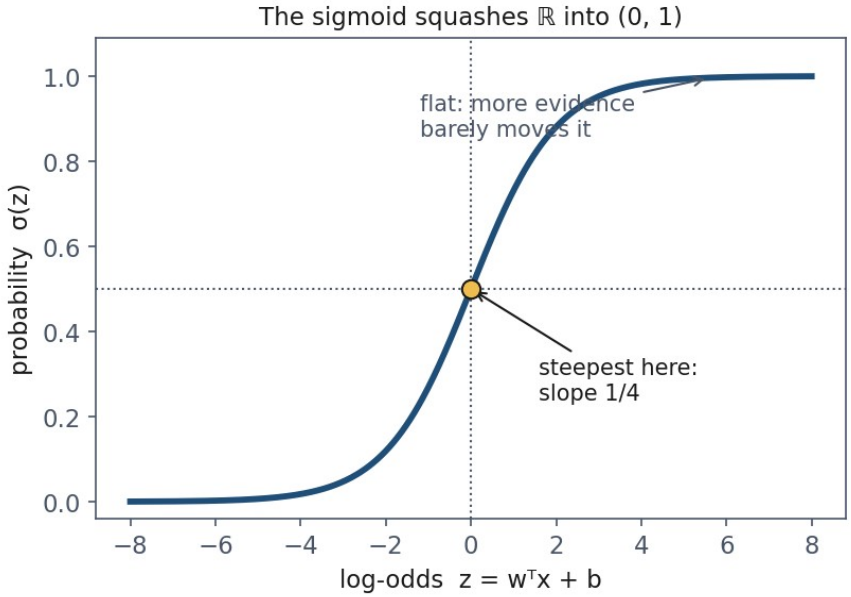
# What goes wrong if we just fit a line

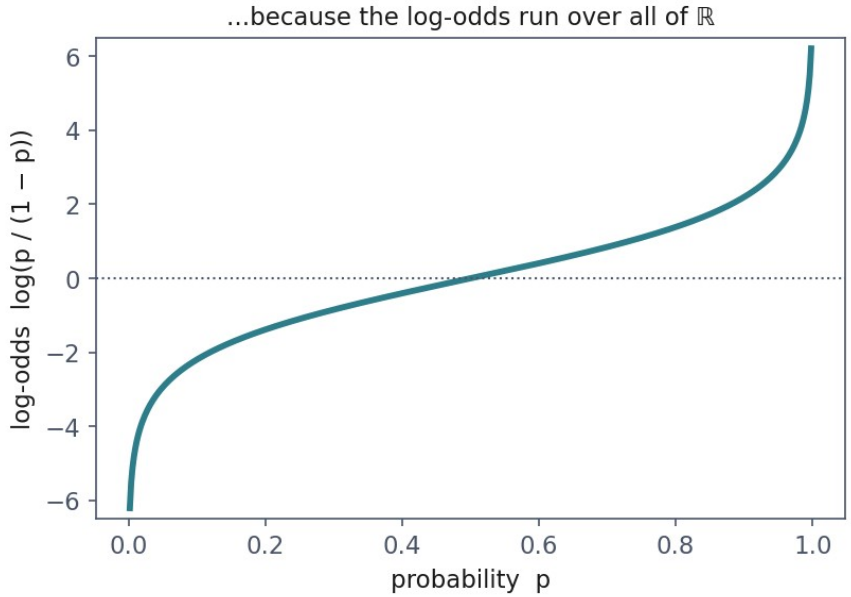





UNIVERSITÀ
DEGLI STUDI
DELL'AQUILA

# The sigmoid






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

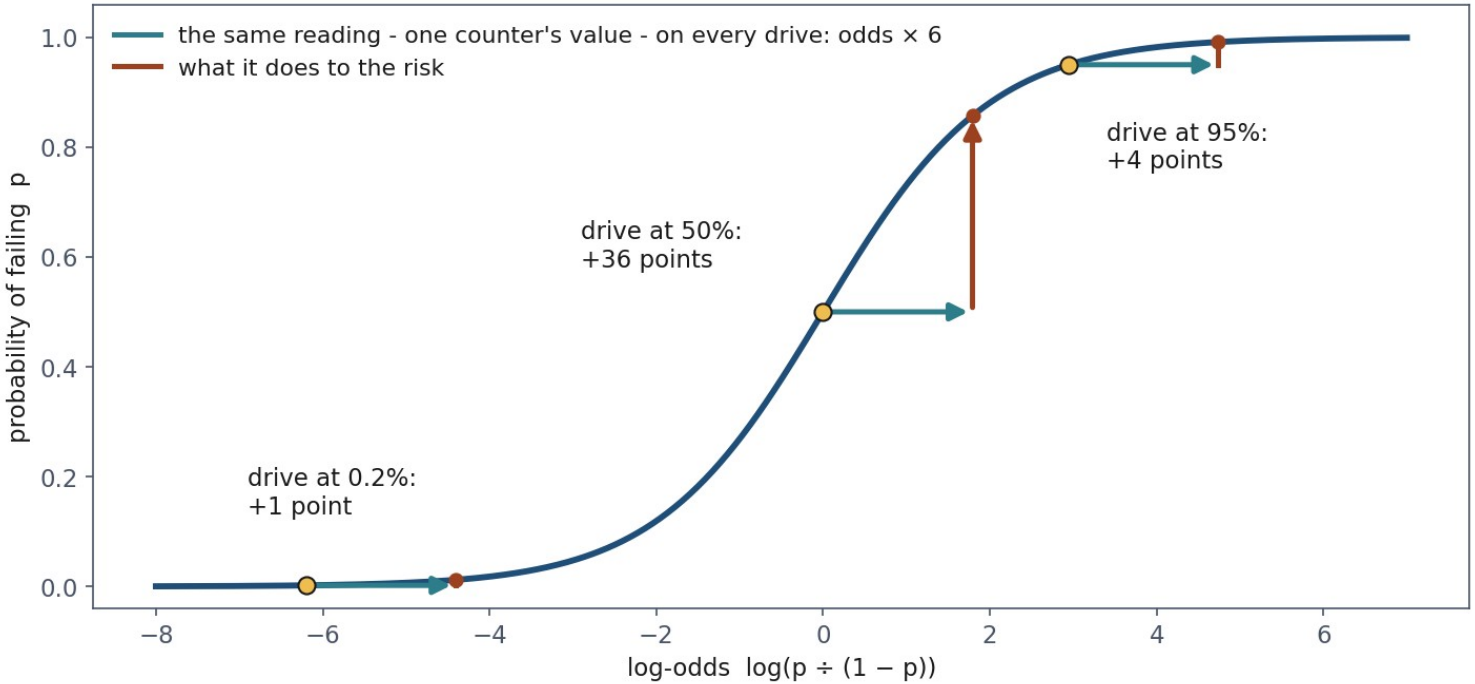
# Why the log-odds

- A line gives any number; a probability, only [0, 1]
- Odds, $p \div (1 - p)$: no ceiling, from 0 to $\infty$
- Log of the odds: no floor either, $-\infty$ to $+\infty$
- $p = 0.99$: $\log(0.99 \div 0.01) = \log 99 = +4.60$; $p = 0.5$: $\log(0.5 \div 0.5) = \log 1 = 0$
- Bonus: on this scale each feature will add a fixed amount, so it will multiply the odds by a fixed factor (slide 14)



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# One reading, three different jumps






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Log-odds of p = P(y = 1 | x): a line

$$\log \frac{p}{1-p} = w^\top x + b \quad \Longrightarrow \quad p = \frac{1}{1 + e^{-(w^\top x + b)}}$$



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# The model, and three properties of the sigmoid

$$P(y = 1 \mid x) = \sigma(w^\top x + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

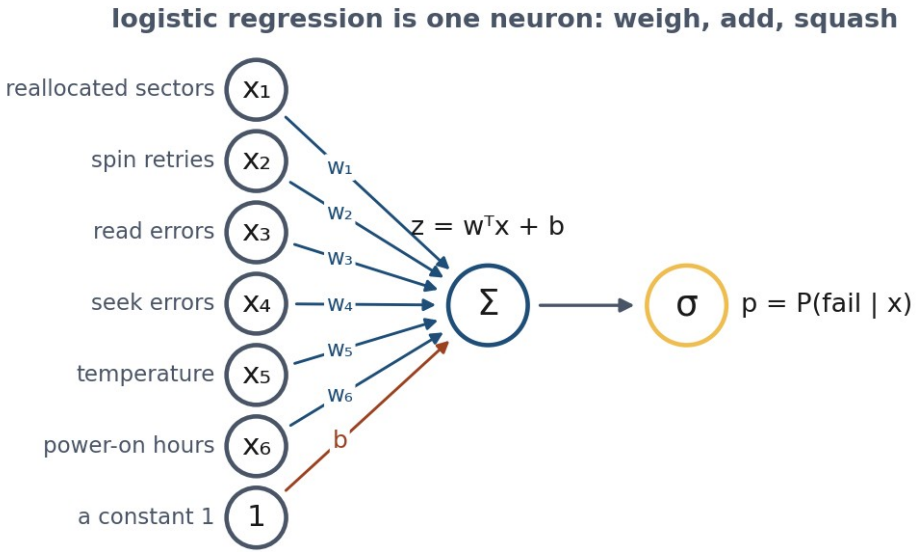
$$\sigma(0) = \frac{1}{2}, \qquad \sigma(-z) = 1 - \sigma(z)$$

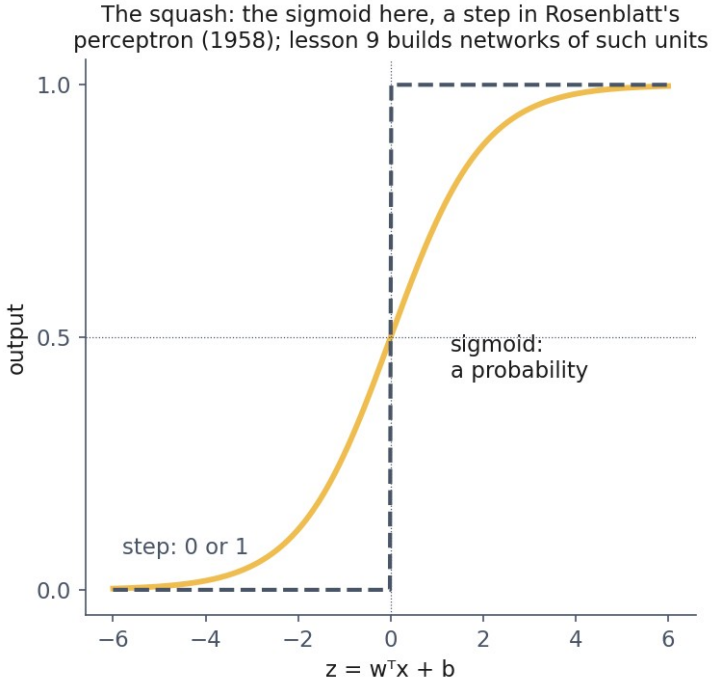
$$\sigma'(z) = \sigma(z)\,(1 - \sigma(z)) \leq \frac{1}{4}$$



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# The model, drawn: one neuron






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# What the coefficients mean

- Coefficients = weights w (`coef_`); intercept b (`intercept_`)
- Standardised (mean 0, SD 1): one SD more of reallocated sectors adds its true coefficient, 1.80, to the log-odds
- log(odds) + 1.80 = log(odds × 6.05): **six times the odds**, slide 10's teal step
- In code: `make_pipeline(StandardScaler(), LogisticRegression())`



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# The truth, and what we recovered

| Feature | True, `disk_data.py` | Fitted, notebook 1 | Odds ×, exp(Fitted) |
|---|---|---|---|
| reallocated_sectors | 1.80 | 1.66 | 5.28 |
| spin_retry_count | 1.15 | 1.17 | 3.22 |
| read_error_rate | 0.85 | 0.87 | 2.39 |
| temperature_c | 0.45 | 0.46 | 1.59 |
| power_on_hours | 0.35 | 0.38 | 1.46 |
| seek_error_rate | **0.00** | **0.02** | **1.02** |



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# The intercept is the average drive’s chance

- Fitted intercept: **−6.09**, the log-odds of a drive average on every counter (all standardised x = 0)
- σ(−6.09) = 1 ÷ (1 + exp(6.09)) = 1 ÷ 442 = 0.0023: a 0.2% chance
- The fleet fails at 3.8% (slide 6): the failures come from the few drives far above average



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Fitting means minimising something, but what?

- Squared error worked for lesson 3
- Why not just apply it to the sigmoid’s output?



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Step 1: each drive gets a p

| Drive | What happened | y | Setting 1: $p = \sigma(w^T x + b)$, the P(fail) |
|---|---|---|---|
| A | failed | 1 | 0.8 |
| B | stayed healthy | 0 | 0.1 |
| C | stayed healthy | 0 | 0.2 |



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Step 2: how likely was the outcome?

| Drive | y | p | Probability of what happened: p if y = 1, 1 - p if y = 0 |
|---|---|---|---|
| A | 1 | 0.8 | 0.8 |
| B | 0 | 0.1 | $1 - 0.1 = 0.9$ |
| C | 0 | 0.2 | $1 - 0.2 = 0.8$ |
| All three together, if the drives are independent: multiply, as for two coins (½ × ½ = ¼) | | | $0.8 \times 0.9 \times 0.8 =$ **0.576** |



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Step 2 as formulas: one drive, then the likelihood

$$P(y \mid x) = p^{\,y}\,(1-p)^{1-y}$$

$$\mathcal{L}(w,b) = \prod_{i=1}^{m} \left(p^{(i)}\right)^{y^{(i)}} \left(1-p^{(i)}\right)^{1-y^{(i)}}$$



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Step 3: two settings, the likelier wins

| Drive | y | Setting 1: p | Setting 1: probability of what happened | Setting 2: p | Setting 2: probability of what happened |
|---|---|---|---|---|---|
| A | 1 | 0.8 | 0.8 | 0.3 | 0.3 |
| B | 0 | 0.1 | $1 - 0.1 = 0.9$ | 0.6 | $1 - 0.6 = 0.4$ |
| C | 0 | 0.2 | $1 - 0.2 = 0.8$ | 0.5 | $1 - 0.5 = 0.5$ |
| Likelihood: the product | | | $0.8 \times 0.9 \times 0.8 =$ **0.576** | | $0.3 \times 0.4 \times 0.5 =$ **0.060** |



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Step 4: minus the log, a cost

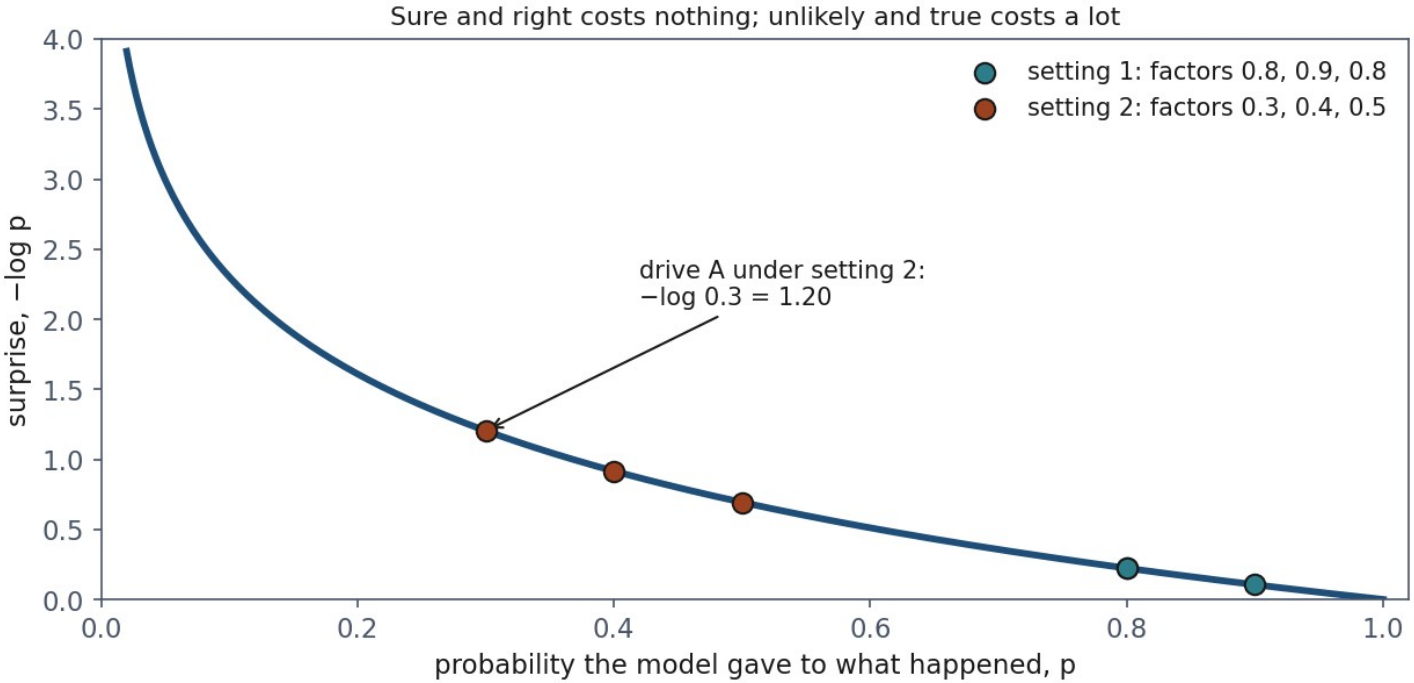
The log turns the product into a sum, one term per drive, and keeps the winner; the minus turns largest into smallest: a cost to minimise, as in lesson 3.

| Drive | y | Setting 1: −log of probability of what happened | Setting 2: −log of probability of what happened |
|---|---|---|---|
| A | 1 | −log 0.8 = 0.22 | −log 0.3 = 1.20 |
| B | 0 | −log 0.9 = 0.11 | −log 0.4 = 0.92 |
| C | 0 | −log 0.8 = 0.22 | −log 0.5 = 0.69 |
| Sum = −log of the likelihood | | 0.22 + 0.11 + 0.22 = **0.55** | 1.20 + 0.92 + 0.69 = **2.81** |



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Surprise: minus the log of a probability






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Cross-entropy, also called log loss

$$L = -[\, y \log p + (1-y)\log(1-p)]$$

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m} L^{(i)}, \qquad p^{(i)} = \sigma(w^{\top} x^{(i)} + b)$$



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Lesson 3’s gradient, $\hat{y} = \sigma(w^T x + b)$

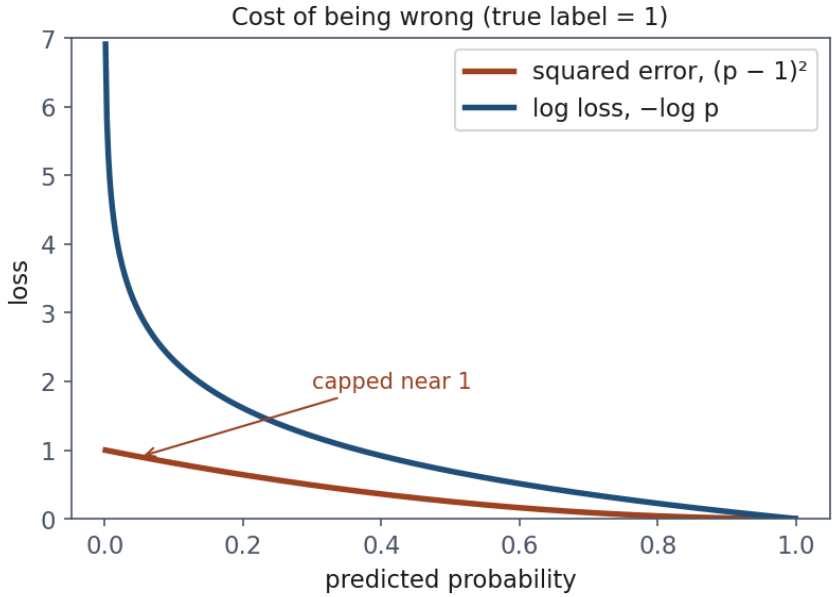
$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} \left( \hat{y}^{(i)} - y^{(i)} \right) x_j^{(i)}$$

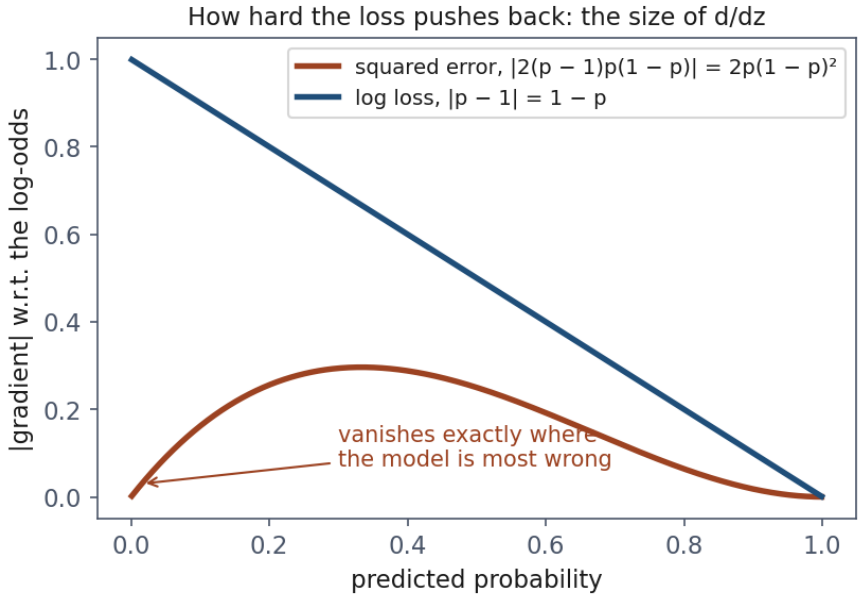
$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} \left( \hat{y}^{(i)} - y^{(i)} \right)$$



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# So why not squared error?






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

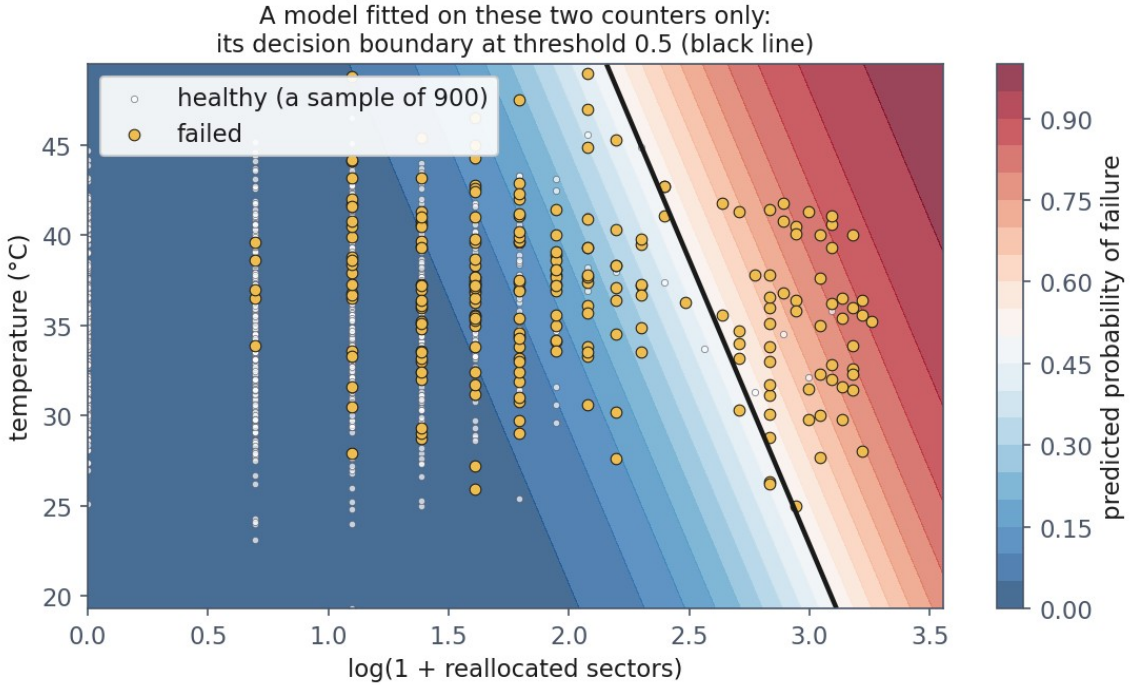
# And it is convex, which squared error is not

- Log loss is **convex** in the weights, so any local minimum is global
- Squared error through a sigmoid is not
- Why not, from slide 26: its slope falls from 0, then climbs back to 0; a convex curve’s slope only grows



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# The decision boundary of a two-counter model






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Notebook 1, live

| section, and its slides | you call (→ the number to come back with) |
|---|---|
| a line on a 0/1 label, 7 | `LinearRegression` → 3,606 negative probabilities |
| odds, and why not squared error, 8-27 | NumPy only: the sigmoid, the two losses, their gradients |
| gradient descent, 25 | `train_test_split`, `StandardScaler`, fifteen lines of NumPy → log loss 0.6931 to 0.0692 |
| the decision boundary, 28 | `make_pipeline(StandardScaler(), LogisticRegression())` on two features |
| the truth, recovered, 15-16 | `LogisticRegression`, `.coef_`, `.intercept_` → seek_error_rate 0.02, intercept −6.09 |

# Break

- Ten minutes



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Two models

| Model, on the 2,000 test drives (76 fail) | Accuracy = right ÷ 2,000 | Failures caught |
|---|---|---|
| Predict “healthy” for everything | every healthy drive: 1,924 ÷ 2,000 = **96.20%** | **0 of 76** |
| Logistic regression, threshold 0.5 | 43 failures + 1,911 healthy: 1,954 ÷ 2,000 = 97.70% | 43 of 76 |

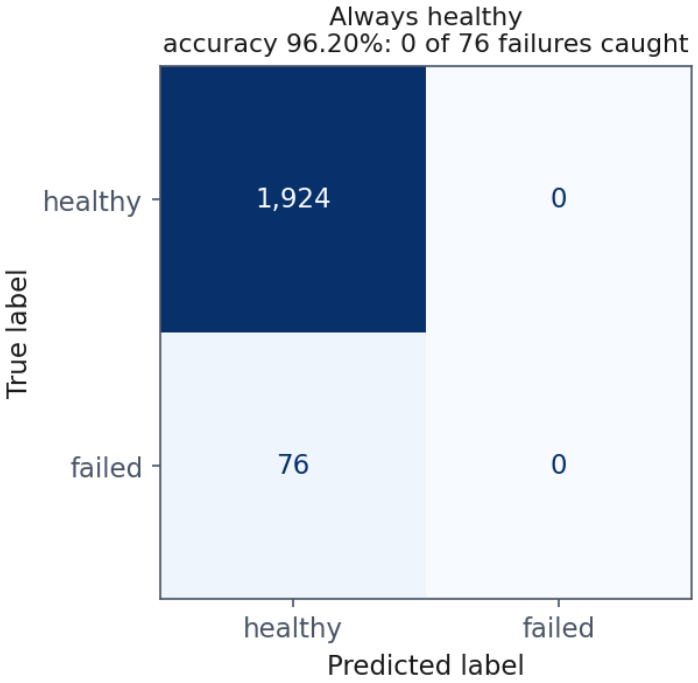


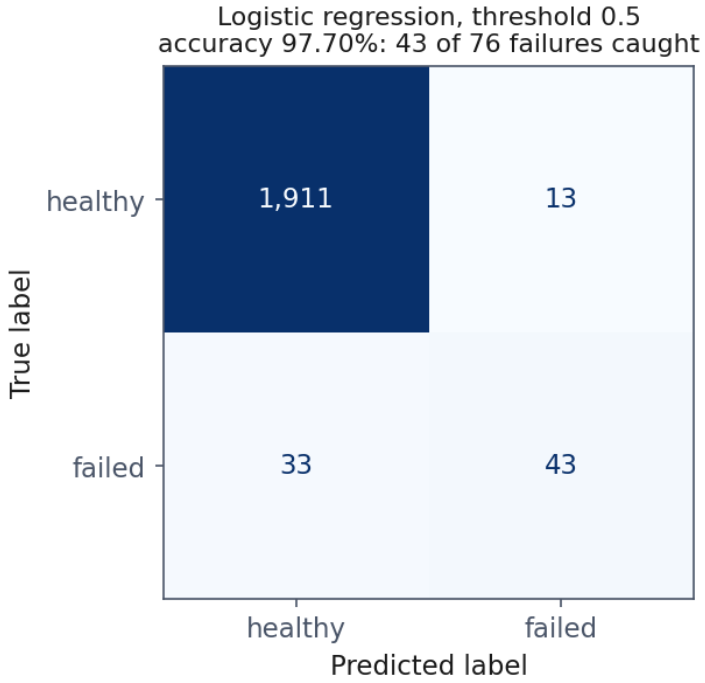
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Why everyone makes this mistake once

- Accuracy is the fraction of predictions that were right
- On a **balanced** problem whose two errors cost the same, that is exactly what you want
- The interesting class is almost always the **rare** one

# Four numbers instead of one






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# The four outcomes, and what they cost

- **False positive (FP)**: we replace a healthy drive. A technician’s hour, €140
- **False negative (FN)**: a drive dies in service. Callout, rebuild, risk, €2,600
- Both costs are assumptions supplied from outside the data, not measured
- **True positive (TP)** and **true negative (TN)**, the right calls, count as €0: each cost is what the error adds



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Precision and recall: look at the denominator

Precision = TP / (TP + FP)
"of the alarms we raised"

| | predicted healthy | predicted failing |
|---|---|---|
| actually healthy | TN<br>1911 | **FP**<br>13 |
| actually failed | FN<br>33 | **TP**<br>43 |

Recall = TP / (TP + FN)
"of the drives that failed"

| | predicted healthy | predicted failing |
|---|---|---|
| actually healthy | TN<br>1911 | FP<br>13 |
| actually failed | **FN**<br>33 | **TP**<br>43 |



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

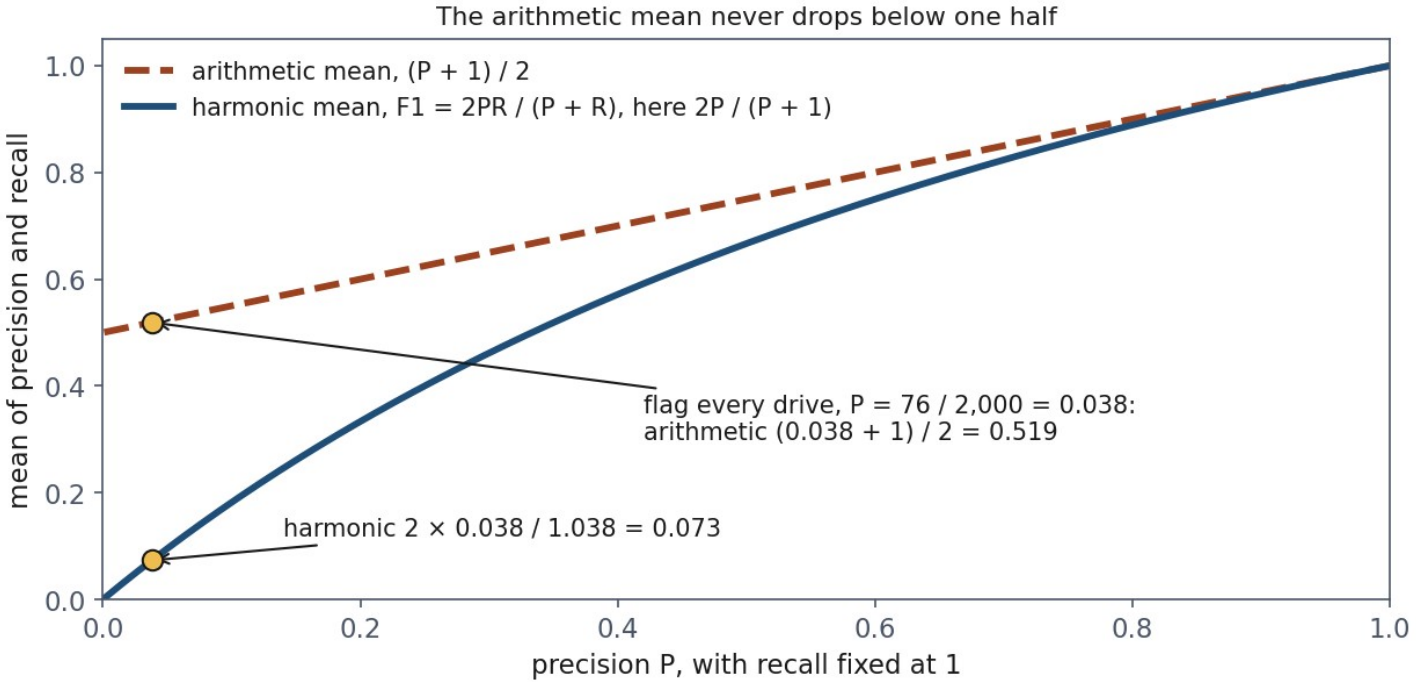
# Our model, in those terms

- True and false positives and negatives: TP, FP, FN, TN
- Precision = TP ÷ (TP + FP) = 43 ÷ (43 + 13) = **0.768**: of the drives flagged
- Recall = TP ÷ (TP + FN) = 43 ÷ (43 + 33) = **0.566**: of the drives that failed
- Specificity = TN ÷ (TN + FP) = 1,911 ÷ 1,924 = 0.993; 1 − 0.993 is the false positive rate (FPR)
- In code: `precision_score`, `recall_score`



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# F1 = 2PR ÷ (P + R): why the harmonic mean






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Read the minority class, not the average

- F1 per class: healthy 0.988 (1,924 drives), failed 0.652 (76 drives)
- `macro avg` F1 = (0.988 + 0.652) ÷ 2 = **0.820**: both classes count the same
- `weighted avg` F1 = (1,924 × 0.988 + 76 × 0.652) ÷ 2,000 = **0.975**: the healthy class dominates
- In code: `classification_report(y_test, y_pred)` prints both rows

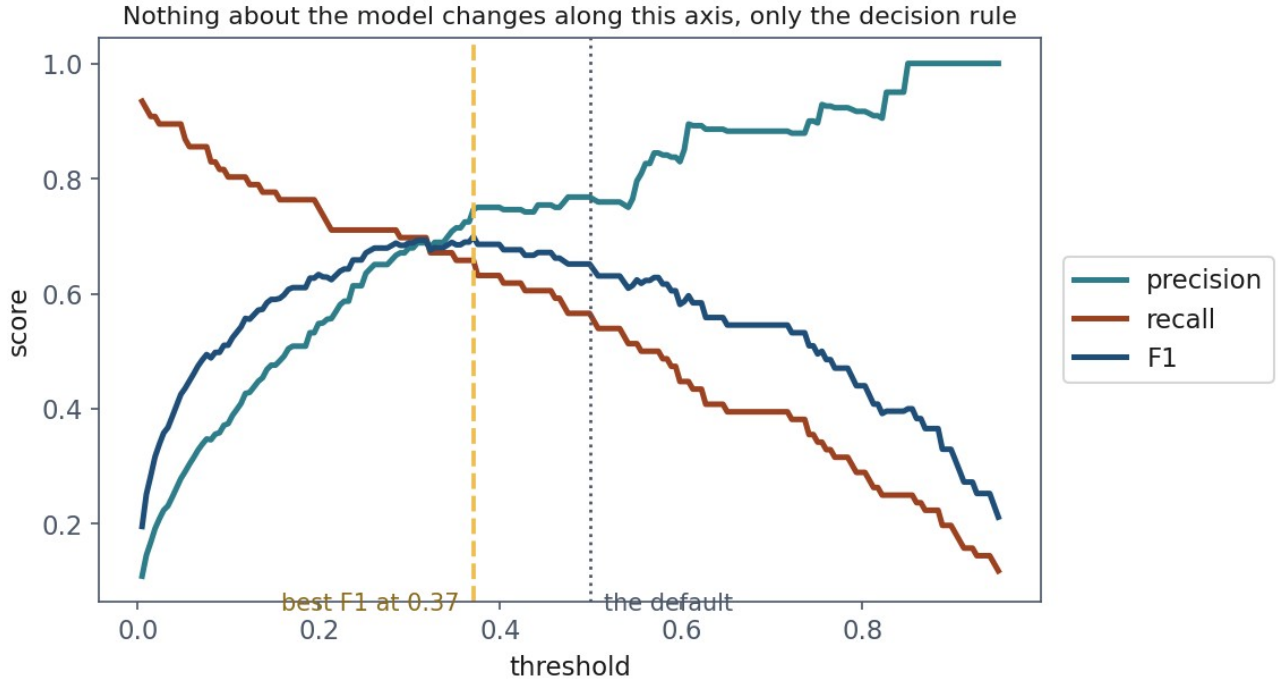


UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Notebook 2, live

| section, and its slides | you call (→ the number to come back with) |
|---|---|
| the number that should worry you, 31 | `accuracy_score` → 96.20% for “always healthy” |
| the confusion matrix, 33 | counts by hand, then `confusion_matrix`, `ConfusionMatrixDisplay` → TP 43 |
| precision and recall, 35-38 | `precision_score`, `recall_score`, `f1_score`, `classification_report` |
| the threshold sweep, 40-41 | `predict_proba`, then the metrics at each threshold |
| the precision-recall curve, 42 | `precision_recall_curve`, `average_precision_score` → 0.717 against 0.038 |

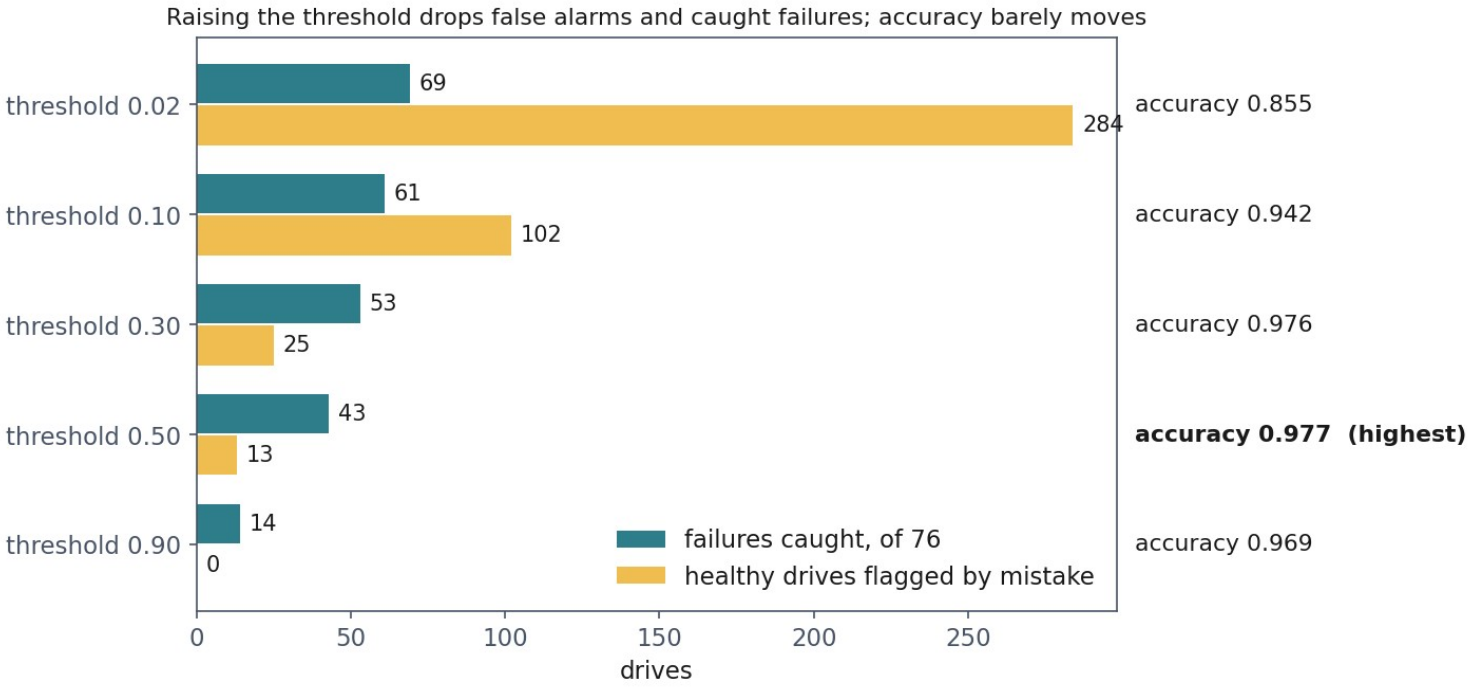
# The threshold is a choice nobody made






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

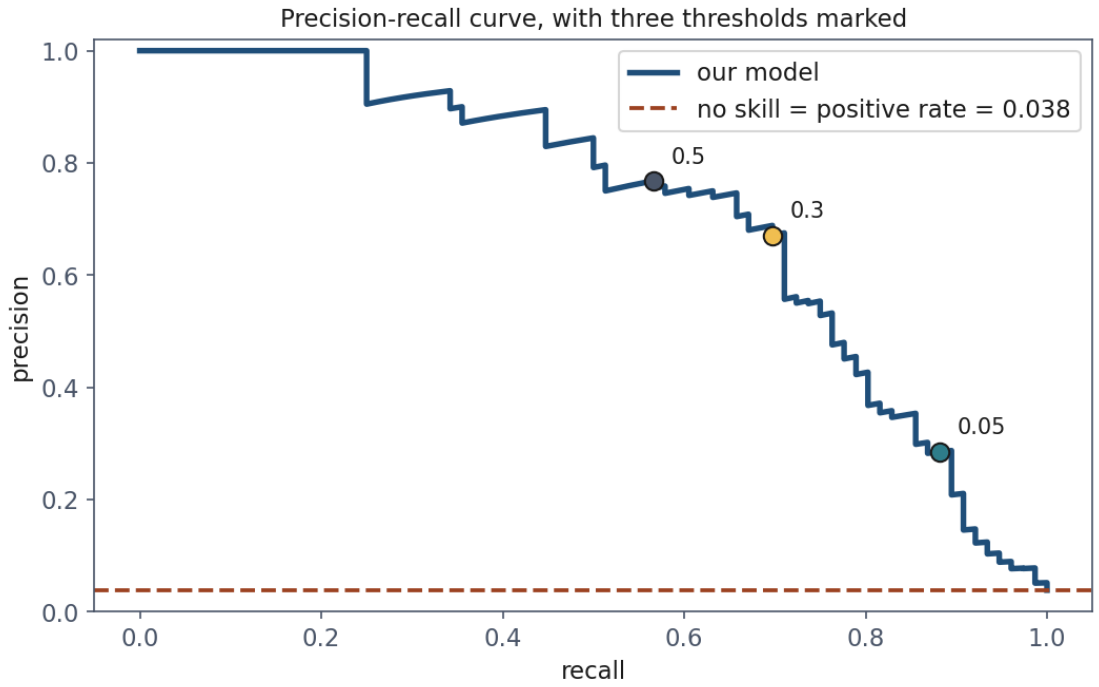
# What the sweep actually costs






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# The precision-recall curve






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

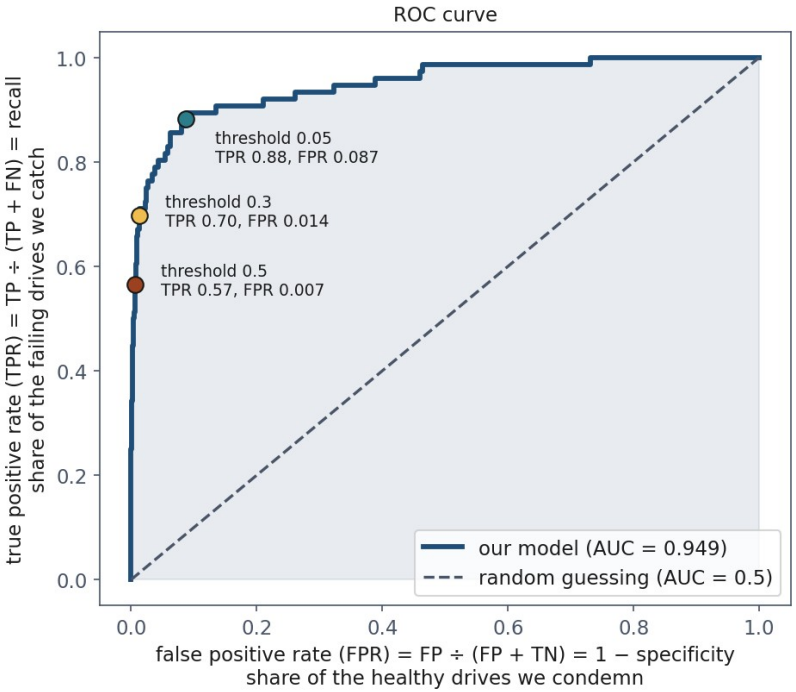
# Comparing models without choosing a threshold

- We keep evaluating at one threshold, but which?
- The **ranking**: the 2,000 test drives sorted by predicted probability, riskiest first
- A threshold is a cut in this list: every drive above the cut is flagged
- What if we score the whole ranking, every cut at once?



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# The ROC curve






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

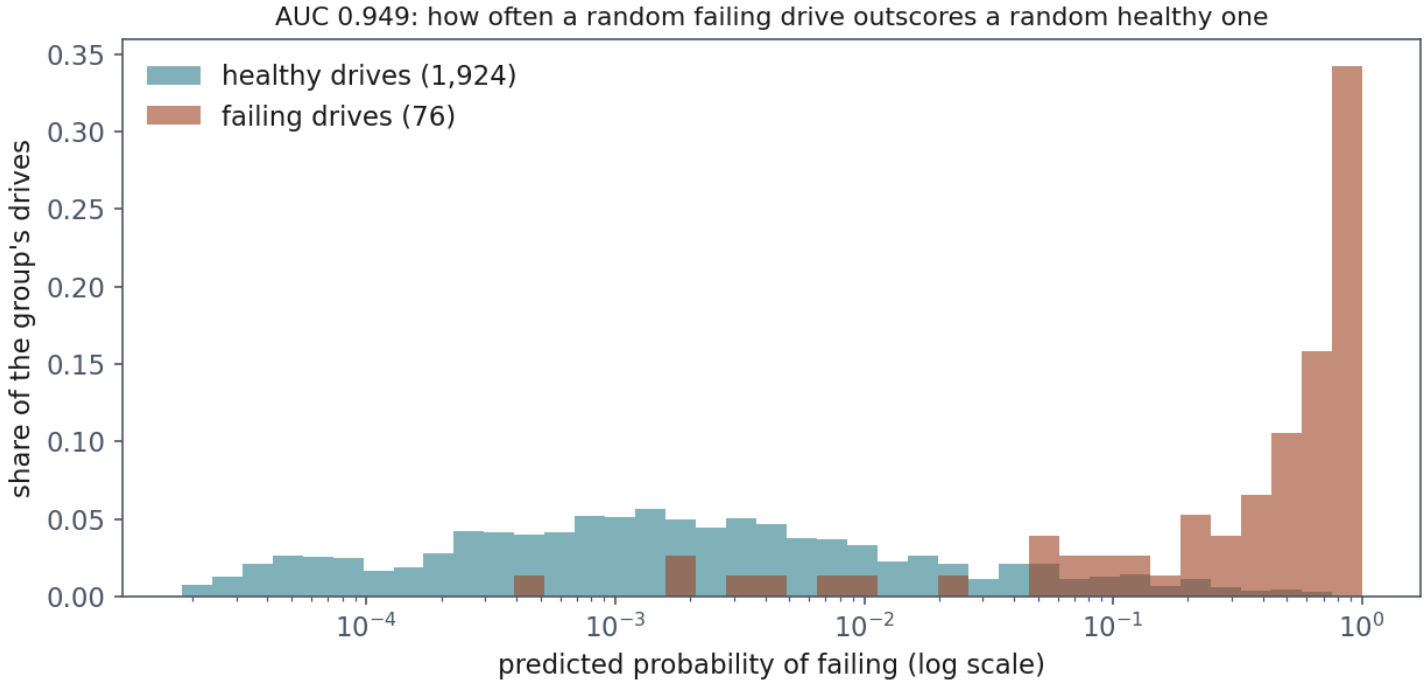
# What the area means

- **Area under the curve (AUC) = the probability that a random failing drive scores above a random healthy one**
- Ours: 0.949 (`roc_auc_score`, notebook 3)
- Check: in 200,000 random pairs, the failing drive scores higher 94.9% of the time
- In code: `roc_auc_score(y_test, scores)`



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# The scores behind the AUC





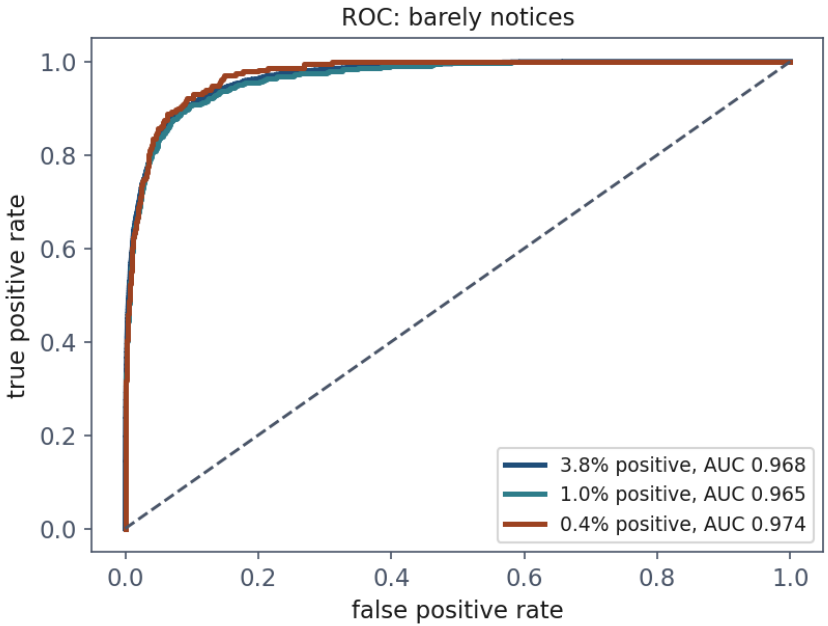

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

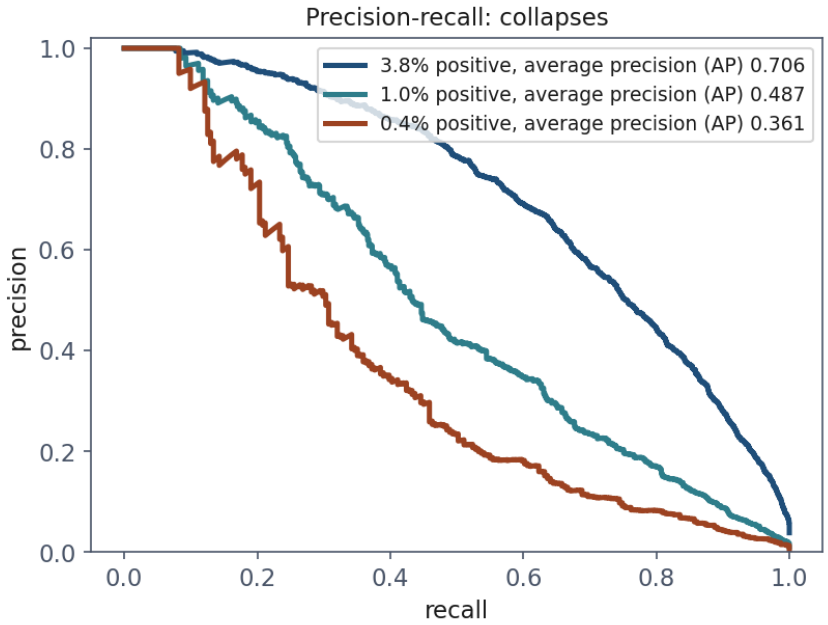
# What AUC does not tell you

- Divide every predicted probability by 10
- AUC is **unchanged**, 0.949: the order of the drives is the same
- Every decision changes: at threshold 0.5, 56 drives flagged before, none after (notebook 3)
- The model is now badly **calibrated**: calibrated means a predicted 0.7 comes true about 70% of the time

# Where ROC misleads

The same model, not retrained, on 60,000 new drives: every healthy drive kept, failures thinned at random






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Why the two curves disagree

- Recall, shared by both curves: TP ÷ (TP + FN), all failures, thinned together: **stays put**
- False positive rate (FPR), the ROC’s: FP ÷ (FP + TN), over **all 57,575 healthy drives**, untouched: 0.0053
- Precision, the other curve’s: TP ÷ (TP + FP); FP **stays 304**, TP falls from 1,127 to 106



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

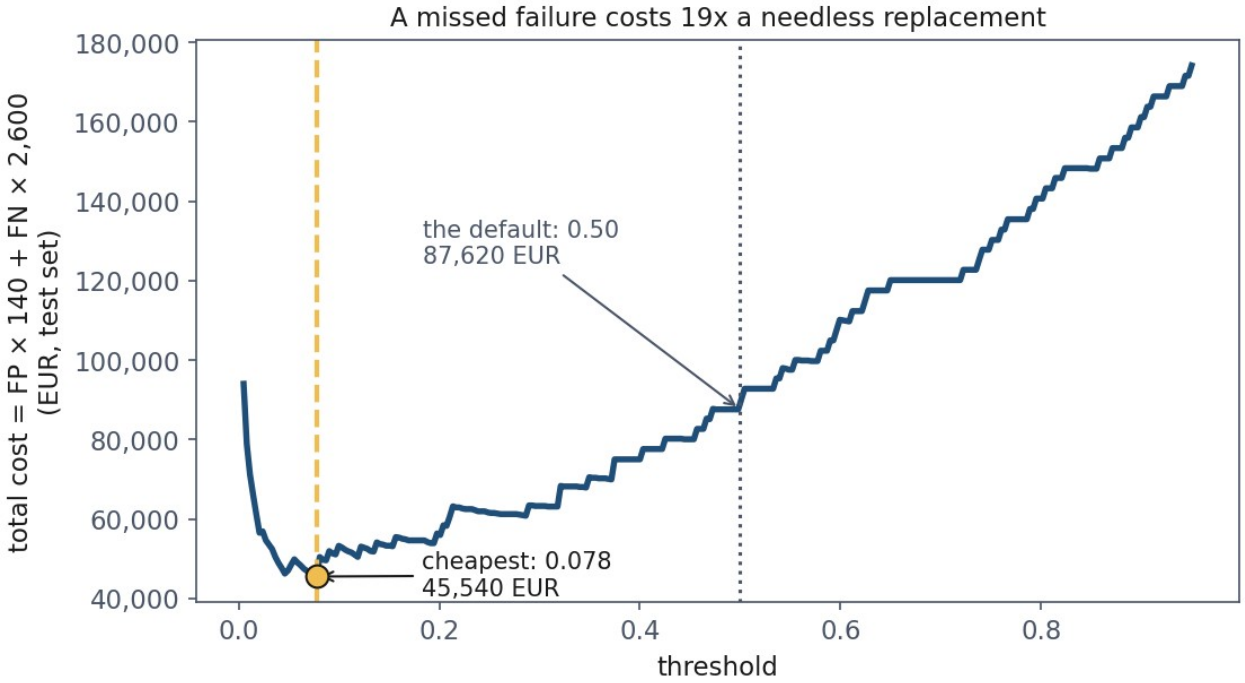
# The rule: report both

- AUC: are failures ranked first? Precision-recall: how many alarms are real?
- On imbalanced data report the **precision-recall curve beside the AUC**
- **Average precision (AP)**: the precision at each failure's place in the ranking, averaged; $\Sigma_k (R_k - R_{k-1}) \times P_k$
- Ours: 0.717 (notebook 2); a random ranking: 76 ÷ 2,000 = 0.038



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

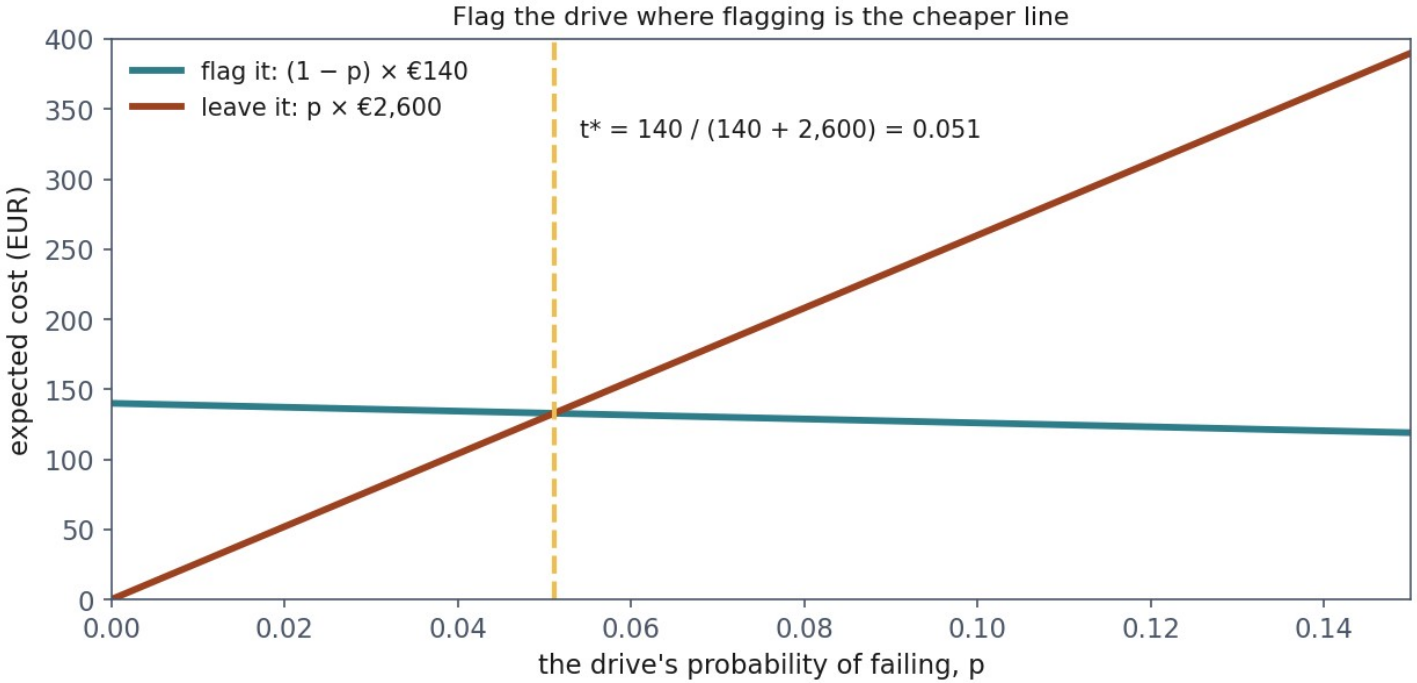
# Choosing the threshold from money






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Where the two costs cross






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Where they cross, in one line

flag if $p\, C_{FN} > (1 - p)\, C_{FP}$

$$\Leftrightarrow \; p > \frac{C_{FP}}{C_{FP} + C_{FN}} = \frac{140}{2740} = 0.051 = t^*$$



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# The caveat you must state

- We chose the threshold **by looking at the test set**: what lesson 1 forbade
- Properly: choose it on a **validation set**, measure once on the test set (lesson 5)
- t* = 0.051, fixed before looking: €49,100 on the test set; 0.078, picked by looking: €45,540
- **Deploy 0.051**: 0.078’s €3,560 was found on the very drives it is measured on

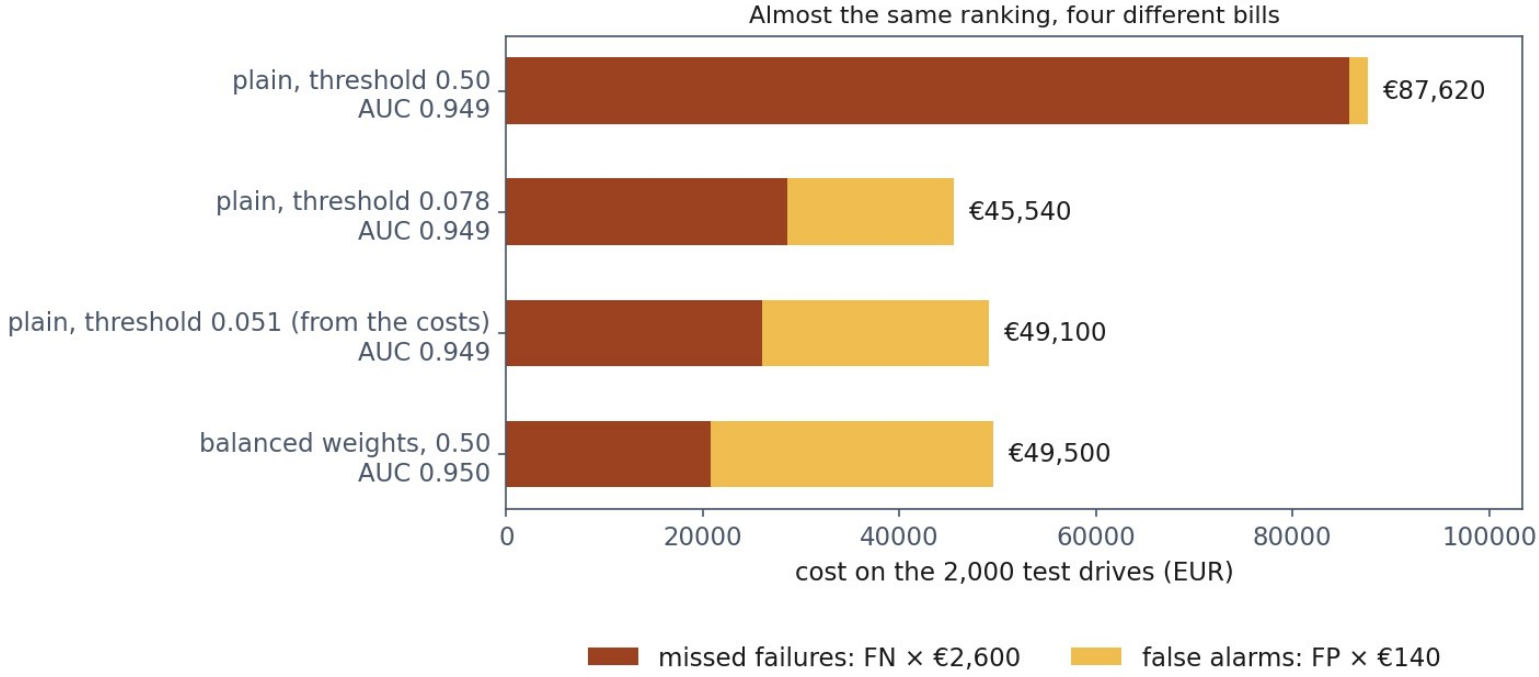
# What class weights do

- Training multiplies each drive’s log-loss term by its **class weight**
- `class_weight="balanced"`: m ÷ (2 × class size), 0.52 healthy, 13.0 failed
- A failure counts as 25 healthy drives: **class frequencies**, not costs
- **In effect it only lowers the threshold**: at 0.5 the weighted model decides like the plain one at 0.038



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Class weights: the same lever, pulled earlier






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Notebook 3, live

| section, and its slides | you call (→ the number to come back with) |
|---|---|
| the ROC curve by hand, 44 | `roc_curve`, `roc_auc_score` → 0.9493 |
| what AUC means, 45-46 | 200,000 random (failing, healthy) pairs → 0.9488 |
| the rare-positive trap, 48-49 | `average_precision_score` beside `roc_auc_score` → AP 0.706 to 0.361, AUC unmoved |
| the threshold from costs, 51-54 | `confusion_matrix` at each threshold → 45,540 EUR at 0.078, against 87,620 at 0.50 |
| class weights, 55-56 | `LogisticRegression` with balanced class weights → AUC 0.950 |
| three classes, 58-60 | `classification_report` → macro F1 0.707, weighted 0.883 |

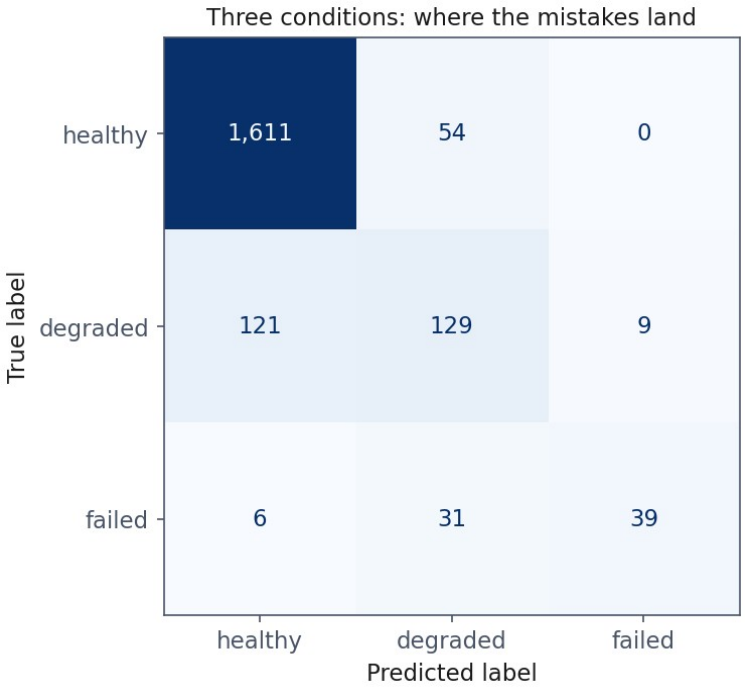
# Three classes: softmax (optional)

- One score per class, $z_k = w_k^T x + b_k$; **softmax**: $P(\text{class } k) = \exp(z_k) \div \Sigma_j \exp(z_j)$: exponentiate, divide by the sum
- A degraded drive: scores 0.615, 0.603, −1.218 → 0.466, 0.460, 0.074, called healthy
- Predicted class = the largest probability, no 0.5 threshold; two classes: the sigmoid of $z_1 - z_0$
- In code: `decision_function` (scores), `predict_proba` (softmax)



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

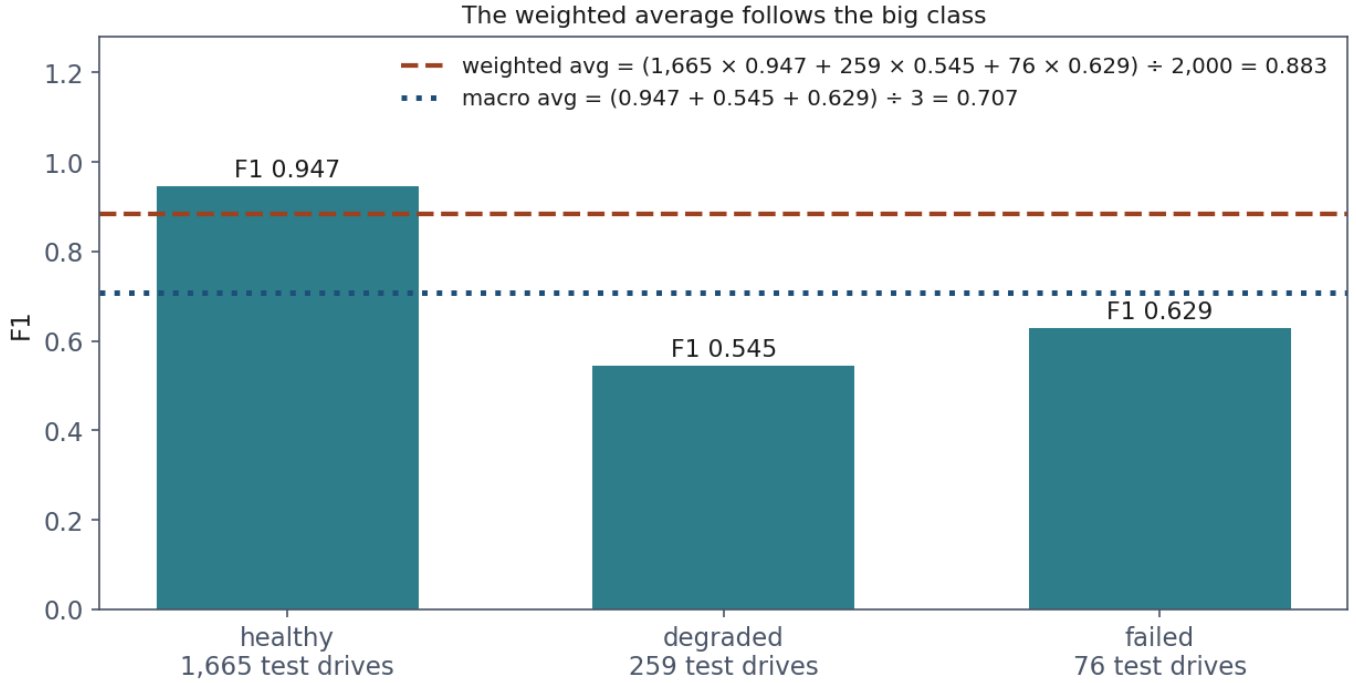
# More than two classes (optional)






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# Macro against weighted, one last time (optional)






UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# What to take away

- Logistic regression: sigmoid of a linear score, fitted by log loss
- **96.20% accuracy, nothing caught**: check the baseline, 1,924 ÷ 2,000
- Precision: the **alarms**; recall: the **failures**
- **Threshold** from costs: 140 ÷ 2,740 = 0.051, if calibrated
- **AUC = ranking**: blind to calibration, hides imbalance

# Homework

- **Exercise 4**, discussed at the start of **Friday 13 November**
- Build a classifier, choose a threshold from a stated cost ratio, defend it